Table of Contents

Visual Python 宣传片录制脚本.....1

一、录制前准备.....1

二、B 站主片：8 分钟逐段脚本.....3

三、60 秒抖音竖屏版.....8

四、镜头与剪辑规范.....9

五、失败预案（现场不要慌）.....10

六、发布信息.....10

七、录制当天 Checklist.....11

八、口播节奏参考.....11

Visual Python 宣传片录制脚本

目标：B 站主片 8 分钟，另剪一版 60 秒抖音竖屏。 形式：真人出镜 + 屏幕录制 + 硬件特写；也可以不露脸，只保留画外音。 语气：第一人称、像在给同事演示，不要念参数，不要喊口号。 核心一句话：**同一套积木范式，既能做 Manim，也能落到 ESP32 上。**

一、录制前准备

1. 三个必须提前跑通的 Demo

Demo	用什么	录制前检查
Manim 主演示	`Hello Manim` 示例	积木能显示、代码能生成、渲染能出片、预览能播放
AI 生成演示	AI 助手 + 一个简单场景	提前跑通过一次，保留“生成前/生成后”两个工程，防止现场模型超时
硬件主演示	ESP32-S3 + 板载 WS2812	串口能连、程序能烧、板

Demo	用什么	录制前检查
		载 RGB 能变色
硬件进阶（可选）	ESP32-S3 + BH1750 + SSD1306 OLED	I2C 能扫到 0x23 和 0x3C，手遮传感器数值会变

建议：Manim 主演示用 `Hello Manim`，因为它只有三块积木，观众一眼能看懂“积木和代码一一对应”。
想要更“数学”的画面，可以额外提前渲染一段线性函数/勾股定理，作为 B-roll 插入，不要在主线里现场搭。

2. 硬件准备

主演示：ESP32-S3 板载 WS2812

- 板子：ESP32-S3（板载 WS2812 一般在 GPIO48，1 颗灯）。
- 线：**USB 数据线**，不是只能充电的线。
- 串口：插上后在设备管理器确认 COM 口；录制时选对端口。
- 固件：提前烧好 MicroPython；不要在现场第一次烧固件。
- 节目效果：灯珠颜色红→绿→蓝→黄，每 0.5 秒一次。

进阶演示：BH1750 + OLED（I2C）

模块引脚	接到 ESP32-S3
BH1750 VCC	3V3
BH1750 GND	GND
BH1750 SDA	GPIO8
BH1750 SCL	GPIO9
BH1750 ADDR	GND（地址 0x23）
OLED VCC	3V3
OLED GND	GND
OLED SDA	GPIO8
OLED SCL	GPIO9

- OLED 常见地址 `0x3C`。
- **接线前先拔 USB**；只接 3.3V，不要接 5V；两块模块共地。
- 如果模块没有上拉电阻，SDA/SCL 各加 4.7k 上拉到 3.3V。
- 不同 S3 开发板的 I2C 引脚可能不同，以板卡丝印为准；平台默认是 SDA 8 / SCL 9。

备用板：ESP32-WROOM / ESP8266

板卡	板载 LED	默认 I2C
ESP32-S3	WS2812，GPIO48	SDA 8 / SCL 9
ESP32-WROOM	GPIO2	SDA 21 / SCL 22
ESP8266-ESP12E	GPIO2，低电平点亮	SDA 4 / SCL 5

3. 屏幕与录音

- 录屏：1920×1080，60fps，CQP 18–20；只录 App 窗口，不录整个桌面。
- App 窗口：1600×1000 或 1920×1080；代码字号 14–16；关闭无关面板。
- 桌面：隐藏图标、任务栏自动隐藏、关闭微信/QQ/浏览器通知。
- 麦克风：单独录，不要用耳机麦；离嘴 15–20cm，稍偏轴，避免喷麦。
- 音量：目标 -16 LUFS，峰值不超过 -1.5 dB；录 10 秒环境底噪，后期降噪。
- 灯光：正面柔光，别让 OLED 反光；拍硬件时稍微俯拍。

4. 录制顺序

先录屏幕主演示，再录硬件特写，最后录真人出镜/口播。
每一段单独录，**不要一镜到底**；每段开头拍一下手或说“第 X 段开始”，方便对轴。

二、B 站主片：8 分钟逐段脚本

时间只是剪辑参考，口播按自然语速来。
每个“操作”后面都有“停 1 秒”，是给观众看清结果，也是给剪辑留气口。

00:00–00:25 | 开场：先给结果

画面 1. 0–4s：全屏播放 Manim 成品「Hello Manim」。 2. 4–8s：切 ESP32-S3 板载 RGB 变色特写。 3. 8–12s：切回 App 分屏，左边积木、右边代码。 4. 12–25s：画面定格在分屏，右下角出小窗真人。

操作 - 不要先放 Logo。 - 第一帧就是成品，第二帧就是硬件。

口播 > 这是一段 Manim 动画。 > > 它不是我手写的 Python，是我用积木搭出来的。 > > 更离谱的是，同一套积木范式，还能直接烧进这块 ESP32。 > > 这个工作台叫 Visual Python。今天我把它的 Manim 和硬件两条链路，完整跑一遍。

字幕/花字 - 积木 → `Manim` - 积木 → `ESP32` - `Visual Python`

备注 - 开场 25 秒内不要讲架构、不要讲理念。 - “同一套积木范式”不要说成“同一套积木”，因为两种工作台的积木并不完全相同。

00:25–01:05 | 问题：为什么做这个

画面 1. 代码编辑器，光标停在 `class HelloManim(Scene)`。 2. 切到命令行/渲染日志，再切回积木。 3. 叠三个关键词：环境、距离、断层。

操作 - 鼠标慢慢从代码移到积木，不要快速甩。

口播 > 我教过也写过不少 Python 入门。初学者卡住的地方，往往不是语法。 > > 第一，代码跑起来之前，环境先劝退一批人。 > > 第二，代码和结果之间隔着一层，学生不知道自己改的那一行到底发生了什么。 > > 第三，从积木到文本，中间有一道断层；很多图形化工具，积木是积木，代码是代码，两边各说各话。 > > Manim 更典型：效果很好，但命令行、场景类、动画对象，对新手不友好。 > > 硬件也是，MicroPython 能跑，但接线、烧录、串口，每一步都能劝退。 > > 所以我想要一个东西：用积木的方式写程序，但右边永远有真实的 Python；既能做数学动画，也能控制开发板。

字幕/花字 - 环境 - 代码 ↔ 结果 - 积木 ↔ 文本

01:05–02:35 | Manim：积木、代码、渲染

画面 1. 示例 → `Manim → Hello Manim`。 2. 切到“图形化和代码”分屏。 3. 点一下“动画场景”积木，右侧代码对应行高亮。 4. 鼠标停在 `Text`、`Write`、`self.wait` 三行。 5. 把字号从 56 改成 96，右侧 `font_size=96` 立刻变化。 6. 点“渲染”，展示渲染日志/进度。 7. 渲染完成，预览播放「Hello Manim」。

操作 - 第 3 步：点积木后停 1 秒，让高亮出现。 - 第 5 步：改字号后停 2 秒，不要马上点渲染。 - 渲染等待时间如果超过 10 秒，后期加速，但保留“正在渲染”的界面。

口播 > 先看 Manim。 > > 左边是积木，右边是实时生成的 Python。这个场景叫 Hello Manim，其实只有三块：创建文字、播放书写动画、等待一秒。 > > 右边就是真正的 Manim 代码：`Text`、`Write`、`self.wait`。不是伪代码，也不是截图。 > > 我点一下这块积木，右边对应的代码会高亮。学生看到的是“我改的是哪一块，代码是哪一行”。 > > 我把字号从 56 改成 96。你看，右边的 `font_size` 立刻变了。 > > 然后点渲染。平台会自动提取场景名、起 Manim 进程、把日志接回来。第一次渲染会慢一点，这里我加速一下。 > > 渲染完，预览直接播。整个过程不需要碰命令行。

字幕/花字 - `Hello Manim` - 积木 = 结构 - 代码 = 文本 - 渲染 = 反馈

备注 - 如果现场渲染失败：不要演“成功”。直接切提前渲染好的视频，口播补一句“这段我提前渲染，逻辑是一样的”。 - 渲染结果建议用全屏，不要只在小预览框里放。

02:35–03:50 | 代码与 AI：不是聊天，是改工作区

画面 1. 切到“代码模式”，展示完整 Python。 2. 切回分屏。 3. 打开右侧「AI 编程助手」。 4. 输入提示词：“生成一个标题从左侧飞入的 Manim 场景，直接改工作区，不要只给代码。” 5. 展示 AI 的工具调用/计划过程（加速）。 6. 结果：工作区出现新积木，右侧代码同步。 7. 点运行/渲染，播放结果。

操作 - AI 生成过程如果超过 20 秒，后期 2–4 倍速，保留“正在调用工具”的关键帧。 - 生成后先停 2 秒，再点渲染。

口播 > 很多图形化工具到此为止：积木是积木，代码是代码。 > > Visual Python 把两边绑在一起：积木改，代码变；代码也能通过 AI 回到积木。 > > 我让 AI 做一件事：生成一个标题从左侧飞入的 Manim 场景。 > > 注意，它不是给我一段代码让我自己贴。它调用工具，先生成完整程序，再由平台的确定性转换器把程序变成真实积木，校验通过后一次性提交到工作区。 > > 这套机制叫“代码优先、积木落地”。AI 写的是代码，但最后落到工作区的是积木；转换不了的语句会报错、给行号，不会悄悄糊过去。 > > 对我来说，AI 的价值不是替我写，而是把“想法 → 可运行的作品”这段路缩短，同时保留可检查的中间结果。

字幕/花字 - 代码优先 - 积木落地 - 可检查的 AI

备注 - 这一段是全片的技术核心，语速放慢。 - 如果 AI 现场失败：切提前录好的“生成前/生成后”，口播说“这里我提前跑了一遍，过程一样”。 - 不要展示 API Key、个人信息、聊天记录。

03:50–04:25 | 插件：平台不是写死的

画面 1. 打开插件管理/插件面板。 2. 展示 Turtle 插件：40 个积木、工具箱、文档。 3. 拖一个 Turtle 积木到画布（可选），不要运行太久。 4. 切回插件列表。

口播 > Manim 之外，这个平台的功能不是写死的。 > > 插件可以装，也可以让 AI 现场做一个。这里我装了 Turtle 插件，多了 40 个积木；插件有自己的工具箱、代码生成和文档。 > > 这一步先不展开，后面单独做一期：从一句需求到一个能用的积木包，AI 怎么规划、编译、验证、安装。

字幕/花字 - 插件 - 40 个积木 - AI 也能造工具

备注 - 这一段是“生态”钩子，不要超过 35 秒。 - 如果没有装插件，改成展示“插件市场/插件列表”，不要假装能点。

04:25–06:05 | 硬件：ESP32-S3，板载 RGB

画面 1. 工程 → 新建工程，四个工作台：Python、ESP32-WROOM、ESP8266、ESP32-S3。 2. 选 ESP32-S3。 3. 工具箱特写：GPIO、PWM、I2C、串口、网络、传感器、执行器、灯光。 4. 示例 → 灯光 → 板载 RGB 循环（或手动搭四块颜色）。 5. 右侧代码特写：`machine`、`neopixel`、`time.sleep`。 6. 插 USB，选 COM 口，点“连接”。 7. 点“运行”。 8. 切板子特写：RGB 红→绿→蓝→黄。 9. 切回 App：输出面板/设备文件。

操作 - 连接串口时停 2 秒，让状态变绿。 - 运行后不要马上切镜头，等灯真的变色。 - 板子特写用手持或俯拍，避免失焦。

口播 > 现在换硬件。 > > 新建工程，四种工作台：Python、ESP32-WROOM、ESP8266、ESP32-S3。我选 S3。 > > 工具箱完全变了：GPIO、PWM、I2C、SPI、串口、Wi-Fi、蓝牙、MQTT，还有传感器、执行器、灯光。 > > 先做一个不用接线的：板载 WS2812。 > > 重复执行里放四块颜色：红、绿、蓝、黄，每块等 0.5 秒。 > > 右边生成的是 MicroPython，不是 Arduino C。 > > 插上 USB，选串口，连接，运行。 > > 你看板子上的灯，它真的在跑。 > > 这时候再看设备文件：`boot.py`、`main.py` 都在板上；也可以直接读文件、传文件、删文件。对教学来说，硬件不再是黑盒。 > > 如果手边没有板子，可以切模拟器，逻辑先跑通，再上真机。

字幕/花字 - `ESP32-S3` - `MicroPython` - 板载 `WS2812` - 代码 → 灯

备注 - 如果灯不亮：先确认板载 LED 型号/引脚；S3 板载 WS2812 一般是 GPIO48。不要现场改接线。 - 如果串口连不上：换数据线、换口、关掉其他串口工具；不要在镜头里反复试 2 分钟。 - 备用方案：切模拟器，或者用 WROOM 板载 LED。

06:05–06:50 | 硬件进阶：光照传感器 + OLED

画面 1. 桌面俯拍：S3、BH1750、OLED、四根杜邦线。 2. 特写：3V3、GND、SDA=8、SCL=9。 3. App：打开提前搭好的“光照显示”工程。 4. 积木：初始化 I2C → 读 BH1750 → OLED 显示文字 → 刷新。 5. 运行。 6. 特写：OLED 显示 lux；手遮住传感器，数值下降；拿开，数值上升。 7. 可选：切到串口输出，展示同样的数值。

口播 > 再往前走一步，接 I2C：BH1750 光照传感器和一块 OLED。 > > S3 默认 SDA 是 8，SCL 是 9，3V3 和 GND 共地。接线之前一定先拔 USB。 > > 积木里初始化 I2C、读光照、OLED 显示、刷新。 > > 运行之后，手一遮，数值掉下来；拿开，数值升上去。 > > 这里有一个细节：平台会在编辑期校验引脚能力，比如你把一个只能输入的引脚拿去做 PWM，它会直接告诉你不行。很多硬件课的坑，在拖积木的时候就挡住了。 > > 从 Manim 到传感器，学生练的是同一件事：把一个问题拆成对象、顺序、参数和反馈。

字幕/花字 - BH1750 - I2C：SDA 8 / SCL 9 - 光照 → OLED - 引脚校验

备注 - 如果 OLED 不亮：先查地址 0x3C、SDA/SCL 是否接反、是否共地；不要现场拆焊。 - 如果传感器读数不动：先用手电筒/遮挡测试，确认不是环境光太稳定。 - 这一段是加分项；如果录制时间紧，可以只保留“板载 RGB”，把传感器放到下一期。

06:50–07:30 | 架构与教学：为什么不是“图形化皮肤”

画面 1. 架构图（四层）。 2. 快速切：Manim 渲染、串口、固件烧录、插件。 3. 叠字：本地、离线、可解释。

口播 > 底层是四层：界面、前端核心、桌面服务、本地运行时。 > > Python、Manim、Pyright、串口、固件烧录都在本地，断网也能用。 > > 对我来说，它不是一个“图形化皮肤”，而是一个可解释的编程环境：积木是结构，代码是文本，运行结果是反馈，AI 是加速器。 > > 这套东西现在有 601 个积木，Manim、NumPy、Pandas、Matplotlib，三类 ESP 开发板，127 个测试文件。 > > 当然它不完美：AI 生成还在迭代，硬件覆盖也有限。但我先把一条完整的路跑通了。

字幕/花字 - 四层架构 - 本地 / 离线 - 601 个积木 - 127 个测试文件

备注 - 数字只作 credibility，不要逐个念。 - 架构图停留 5 秒以上，给观众看清。

07:30–08:00 | 结尾：给谁用，下一期讲什么

画面 1. 左边 Manim 成品，右边 ESP32 灯。 2. 切品牌落版：Visual Python / 不会 Python，也能做 Manim。 3. 右下角真人小窗。

口播 > 如果你也在教编程、做数学可视化，或者想给硬件课找一个不那么劝退的入口，可以试试 Visual Python。 > > 项目地址和文档我放在简介。下一期，我讲 AI 怎么从一句话生成一个能用的积木插件。 > > 我是谭伟，我们下期见。

字幕/花字 - Visual Python - 项目地址见简介 - 下期：AI 造插件

备注 - 结尾不要突然黑屏；留 2 秒品牌落版。 - B 站可以加“一键三连”口播，但不要放在技术段落中间。

三、60 秒抖音竖屏版

9:16，1080×1920；语速比 B 站快 15%；字幕 2 行以内；前 3 秒必须给结果。

时间	画面	口播
0–3s	Manim 成品 + ESP32 灯快切	这是用积木做的 Manim，还能烧进 ESP32。
3–10s	分屏：左积木、右代码	左边拖积木，右边就是真正的 Python；改一个字，代码和画面一起变。
10–18s	改字号 56→96，代码高亮	不用命令行，点一下渲染。
18–25s	渲染结果「Hello Manim」	Manim 自己跑，预览直接播。
25–35s	新建 ESP32-S3、板载 RGB	同一套积木范式，换成 ESP32-S3，连上串口，板载 RGB 直接跑。
35–45s	手遮 BH1750，OLED 数值变化	再接一个光照传感器和 OLED，手一遮，数值就变。
45–52s	AI 助手生成积木	AI 还能把代码落成积木，不是只会在聊天框里写代码。
52–58s	插件列表/Turtle	插件也能自己长。
58–60s	品牌落版	Visual Python，让数学动起来，也让硬件不再劝退。

抖音标题 > 不会 Python，也能做 Manim？我还把它烧进了 ESP32

抖音话题 > #编程 #Python #Manim #ESP32 #图形化编程 #创客 #数学可视化

封面 - 上行大字：不会 `Python` - 下行大字：也能做 `Manim` - 小字：还能烧进 `ESP32` - 背景：App 分屏 + 板载 RGB 特写（左右拼）

四、镜头与剪辑规范

1. 屏幕录制

- 只录 App 窗口，不录桌面；分辨率 1920×1080，60fps。
- 鼠标移动要慢；点击前停 0.3 秒，点击后停 1 秒。
- 不要频繁切窗口；需要切代码/积木时，用同一窗口内的模式按钮。
- 代码字号 14–16；积木缩放到能看清字段；AI 面板不用时关掉。
- 录制前清空输出/问题面板，避免旧报错入镜。

2. 真人出镜（可选）

- 半身，眼平，50mm 左右；背景干净，不要背光。
- 说话看镜头，不要看屏幕；演示时再看屏幕。
- 穿纯色衣服，避免细条纹（摩尔纹）。
- 小窗放在右下角，不要挡字幕和代码。

3. 硬件特写

- 俯拍 45°，柔光从侧面打，避免 OLED 反光。
- 先对焦灯珠/屏幕，再开机；不要手持晃动。
- 拍一组空镜：USB 插入、板子灯亮、手遮传感器、OLED 数值变化。
- 线材用短线，颜色区分：红 3V3、黑 GND、黄 SDA、白 SCL。

4. 剪辑

- 开头 3 秒内必须出现成品或硬件动作。
- 段落之间硬切；只有章节之间用 0.2s 淡入淡出。
- 关键代码/数值加 1.3–1.6 倍数字变焦，配 1px 高亮框。
- 字幕：1080p 下 28–32px，最多两行；关键词用青色 `#2DD4BF`。
- BGM：低频氛围，音量压到 -28 dB 左右，人声出现时自动 duck。
- 音效：点击、渲染完成、灯变色可以加轻量 whoosh/pop，不要盖过人声。

5. 录音

- 单独录，48kHz/24bit；离嘴 15–20cm，稍偏轴。
- 每段先录 3 秒静音，后期采样降噪。
- 口误不要停整段，停半句，重读那一句；剪辑点放在句间。
- 语速：B 站 240–260 字/分钟；抖音 300 字/分钟左右。
- 句尾往下压，不要每句都上扬；技术词说清楚，不要吞字。

五、失败预案（现场不要慌）

问题	处理
Manim 渲染失败	切提前渲染好的 MP4，口播“这段我提前渲染，逻辑一样”；不要演成功
渲染太慢	录制时保留“正在渲染”，后期 4–8 倍速；不要剪掉所有等待
AI 超时/报错	切提前录好的生成前后对比；口播“模型有波动，这里用我提前跑的结果”
串口连不上	换数据线/换 USB 口/关其他串口工具；仍不行切模拟器
板载灯不亮	确认板卡型号和 LED 引脚；S3 板载 WS2812 一般 GPIO48；不要现场改接线
OLED 不亮	查 3V3/GND、SDA/SCL、地址 0x3C；先用手电照屏看是否上电
传感器数值不动	用手遮/手电测试；确认 ADDR 接地（0x23）
麦克风有底噪	录 10 秒环境音，后期降噪；不要用耳机麦
桌面弹出通知	开录前开“专注模式”，关闭微信/QQ/浏览器通知

六、发布信息

B 站

标题 > 我用积木做 Manim，还把它烧进了 ESP32 | Visual Python 开发实录

备选标题 > 不会 Python，也能做 Manim？我做了一个能连硬件的图形化工作台

简介 > 用积木做 Manim，是一种什么体验？这期我把 Visual Python 的 Manim 工作台和 ESP32-S3 硬件链路完整跑了一遍：积木 → Python → 渲染 → 串口 → 传感器。 > > 00:00 开场 > 00:25 为什么做 > 01:05 Manim 演示 > 02:35 代码与 AI > 03:50 插件 > 04:25 ESP32-S3 > 06:05 光照传感器 + OLED > 06:50 架构与教学 > 07:30 结尾 > > 项目还在迭代，欢迎在评论区告诉我你最想看的下一期。

标签 > 编程，Python，Manim，图形化编程，ESP32，MicroPython，创客教育，数学可视化

抖音

标题 > 不会 Python，也能做 Manim？我还把它烧进了 ESP32

简介 > 用积木做 Manim，还能一键烧进 ESP32。Visual Python，让数学动起来，也让硬件不再劝退。

话题 > #编程 #Python #Manim #ESP32 #图形化编程 #创客 #数学可视化

七、录制当天 Checklist

- ☐ 桌面图标/通知/任务栏已隐藏
- ☐ App 窗口 1600×1000 或 1920×1080，代码字号 14–16
- ☐ Manim 示例能渲染、能预览
- ☐ AI 生成 Demo 提前跑通，保留前后工程
- ☐ ESP32-S3 已烧 MicroPython，串口能连
- ☐ 板载 RGB 示例能跑，灯能变色
- ☐ BH1750 + OLED 已接好，地址 0x23 / 0x3C 能扫到
- ☐ 麦克风录 10 秒底噪，监听无爆音
- ☐ 每段开头拍手/报段号
- ☐ 录完立即备份素材，不在原工程上覆盖
- ☐ 导出前检查：无个人信息、无 API Key、无聊天记录

八、口播节奏参考

- 开场：短句，每句 5–10 个字，留白。
- 演示：一句话只讲一个动作；“你看”“注意这里”后面停 1 秒。
- 技术解释：先给结论，再给原因，不要反过来。
- 硬件：先讲“会发生什么”，再操作；灯亮/数值变化后停 2 秒，让观众自己看。
- 结尾：不要总结全片，给下一个问题。

如果只能记一句话：**先让观众看到结果，再解释你是怎么做到的。**